Resprox — Knowledge Transfer

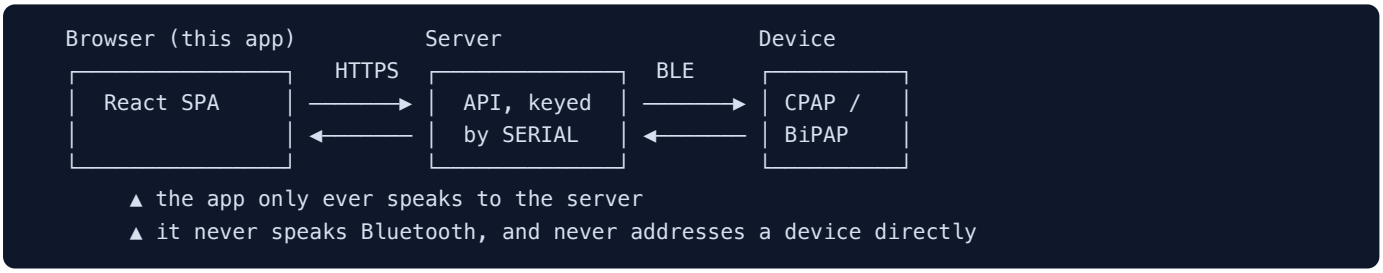
Device data files, the settings pipeline, and user-facing messaging — end to end.

CPAP/BiPAP compliance dashboard (`cpap-dashboard` / RESPROX) · React 18 + Vite · Document revision 1.0

Contents

1. System boundary — what talks to what

This is the single most important thing to understand before changing anything.



The web app communicates **only with the backend server**, and everything is addressed by **device serial number**. The server owns the conversation with the physical device, including the BLE hex protocol. The browser has no Bluetooth code and must not acquire any.

> **Practical consequence.** When a screen needs to change a device setting it `PATCH`es the server and then polls a command status. It does not build device frames, and it cannot know the device applied a change until the server says so.

There are two independent data paths into the app:

Path	How data arrives	Used by
Live telemetry	Server API over HTTPS, per serial. Device pushes to the server; the app pulls.	Dashboard, Therapy, Trends, Reports, Settings
Offline file	An operator copies `REPORTFL.TXT` off the device's SD card / USB and uploads it in the browser. Parsed locally; never sent anywhere.	Upload Data screen (`/admin/upload`)

2. Device data file: accepted formats

Property	Accepted
Canonical filename	`REPORTFL.TXT` (the name is not enforced — content is what matters)
File picker filter	`.txt` , `.TXT` , `.csv` , `.log` , `text/plain`
Maximum size	**8 MB**. Larger is refused with the size stated back to the user.
Encoding	Plain text, read via the browser's `File.text()` (UTF-8).
Line endings	`LF` or `CRLF` — both handled.
Blank lines	Ignored silently (the device writes many).
Record shape	Comma-separated, positional. Opens with `S` , closes with `A` , and most firmware appends the serial after the terminator.
Minimum fields	**39** (through the leak value). The terminator and serial are optional, so older firmware without a trailing serial still parses.
Transport	None. Parsing happens entirely in the browser; the file is never uploaded to any server.

A typical record — this is the reference example from the device specification:

```
S,22,12,23,22,12,23,4,554,20,4,0,0,163,0,215,339,0,0,14,28,15,30,5,0,2,0,39,0,3,4,3,0,2,1,1,1,6,8,A
```

and the same shape as emitted by current firmware, with a serial appended:

```
S,15,6,24,15,6,24,4,901,20,4,0,0,2,0,216,554,0,0,16,26,17,1,5,0,2,0,45,0,0,0,1,0,2,1,1,255,3,123,A,24000378
```

3. The record specification (41 fields)

Index 0 is the record marker. Both known firmware variants are supported: 40 fields (ending at the `A` terminator) and 41 fields (terminator plus serial).

Index	Field	Encoding	Notes
`0`	Record marker	`S`	Anything else → line rejected
1–3	Start day, month, year	uint8; 2-digit year → 2000+	Calendar-validated
4–6	End day, month, year	uint8; 2-digit year → 2000+	Lets a session cross midnight
7	Mode	uint8	Observed 3 and 4; the pressure columns are the reliable signal (see below)
8	Volume transfer	uint16	Retained, not currently reported
9	Max pressure	uint8 cmH_2O	Fixed set pressure when min is 0
10	Min pressure	uint8 cmH_2O	> 0 with max > min → AUTO
11–12	IPAP, EPAP	uint8 cmH_2O	Either > 0 → BiPAP
13	Pressure change count	uint8	Retained
14	CPAP/BiPAP counter	uint8	Retained
15–16	Air flow, tidal volume	uint16	Shown in CSV export
17–18	Resp rate, start EPAP	uint8	Observed 0 across all sample data
19–20	Start hour, minute	uint8, 0–23 / 0–59	Range-validated
21–22	End hour, minute	uint8, 0–23 / 0–59	Range-validated
23	Humidifier level	uint8	Carried into report settings
24	Ramp time	uint8 minutes	Carried into report settings
25–26	Auto on/off, titration factor	uint8	Retained
27	Rate change factor / minute ventilation	uint8	Meaning depends on mode
28	CSA count	uint8	Summed per night → AHI numerator
29	OSA count	uint8	
30	HSA count	uint8	
31–32	Reserved	uint8	Undocumented; 32 is always 0 in sample data
33–34	A-Flex on/off, level	uint8 (2 = on)	Carried into report settings
35	Reserved	uint8	Undocumented
36	Mask type	1 Nasal, 2 Full Face, 3 Pillow	`255` seen on older firmware = unset
37	Reserved	uint8	Undocumented
38	Leak	uint8 L/min	**See §6 — values ≥ 120 are status codes, not**

39	Terminator	`A`	Optional; recorded but not enforced
40	Serial number	text	Optional; drives the serial shown on screen

Mode classification

The numeric mode byte is not trusted across firmware revisions. Mode is derived from the pressure columns, which have been consistent:

Condition	Result
IPAP > 0 or EPAP > 0	**BiPAP** — set pressure taken as IPAP
min > 0 and max > min	**AUTO CPAP** — reported as a min–max band
otherwise	**CPAP** — fixed pressure taken from the max column

4. Validation & rejection rules

Parsing never throws. A bad line is collected with a reason and reported on screen; the rest of the file still parses. There are two distinct outcomes — *rejected* and *excluded*.

Rejected — the line is not a record at all

Message shown	Trigger
does not start with the "S" record marker	First field is not `S`
only *N* fields, expected at least 39	Truncated line
unreadable start date/time	Non-numeric, or a date the calendar rejects (month 13, day 32, hour 24)
unreadable end date/time	As above, on the end timestamp

These appear on screen as *“**N** line(s) skipped”*, each with its line number and a 48-character excerpt, capped at 5 with a “Show all *N*” toggle.

Excluded — a valid record that cannot be a night of therapy

Rule	Why
End timestamp precedes the start	Impossible; almost always a device clock reset
Duration > 24 hours	Not a single night; counting it inflates usage totals

Excluded records are kept and counted, but contribute to no statistic. The screen states *“**N** session(s) excluded from the statistics”* and explains why. The per-night table shows a red `–N` beside the session count for affected days.

> **Design principle.** Nothing is silently dropped. Every record is either counted, or visibly reported as skipped/excluded with a reason. A number on this screen can always be traced back to the lines that produced it.

5. From records to nights: the aggregation

One line is one *session* (a mask-on period). One row in the report is one *night*. The parser performs that collapse, deliberately, before the report builder sees the data.

> **Why here and not in the report builder.** The report builder keeps a single frame per calendar day, which is correct for live telemetry (one frame per night) but would throw away all but the last session of a day in a file that records every mask-on separately. Aggregating in the parser makes the builder's de-duplication a no-op.

Quantity	Rule
Night attribution	A session belongs to the date it **started** on. A 23:35 → 01:50 session is one night, counted as 2 h 15 m, on the earlier date. This is the standard sleep-therapy convention.
Usage hours	Sum of all included session durations that day.
AHI	(CSA + OSA + HSA summed) ÷ usage hours. Zero usage → AHI 0.
Leak	**Duration-weighted** mean of valid readings. Falls back to a plain mean if every contributing record has zero duration; `null` if none are valid.
Set pressure, tidal volume, air flow	Duration-weighted means.
Compliance	A night passes at **≥ 4 hours** of total usage.

Duration weighting matters: the device writes short configuration frames alongside real sessions, and an unweighted mean would let a two-minute frame pull a night's average as hard as an eight-hour one.

6. Known data quirks you must know about

6.1 The leak field is not always a leak ACTION NEEDED

Field 38 is bimodal. Plausible readings sit in **0–30 L/min**. Alongside them the devices emit codes at **120–129** and **196** — the 196 frames are the zero-duration configuration echoes written at power-on.

In one 378-record sample, **328 of 375** counted records carried such a code. Read as litres per minute they would report almost every night as a blown mask. The mean of the plausible values is 7.4 L/min, which is clinically normal.

Current behaviour: anything **≥ 120** is treated as a status code — carried through as raw data, excluded from the leak average and the leak chart, and counted for the operator. The screen names the exact codes found so they can be taken to the firmware team.

> **This needs confirming with firmware.** If 120–129 actually encodes a real leak (an offset? tenths?), the rule is a one-line change in `reportFileParser.js` (`LEAK_SENTINEL`). Until then, leak statistics are based only on values below 120.

6.2 Set pressure is a setting, not a measurement

The file records the pressure the device was *told* to deliver. There is no measured pressure trace. Reporting that value under “median” or “95th percentile” would misstate it, so the report takes a `pressureSource: 'set'` flag which:

- titles the chart **Set Pressure (cmH_2O)** rather than Pressure;

- leaves median, 95th percentile and maximum blank (“—”);
- reports the mean as **Set Pressure (avg)**;
- adds a sentence to the report footnote explaining the omission.

The live-telemetry path passes no flag and is completely unchanged.

6.3 Metrics the device does not transmit

The apnea/hypopnea split (AI/HI), central/obstructive/unknown apnea indices, daily median pressure, respiratory rate and minute ventilation are not present. They render as “—” and their sections stay hidden. A blank is correct where an estimate would be invented.

7. The upload screen, step by step

Route `/admin/upload` · sidebar entry **Upload Data**.

1. **Drop or browse** for the file. Drag-and-drop and click-to-browse both work; the drop zone is keyboard reachable.
2. **Parse.** Runs synchronously in the browser. On success the file chip shows name, size, record count and day count.
3. **File Contents.** Six tiles — Sessions, Days used, ≥ 4 hours, Total usage, AHI, Mode — plus any data-quality warnings (§4, §6).
4. **Report Details.** Serial (pre-filled from the file), date range (pre-filled to the file's full span), and optional patient name, ID, date of birth, gender and device model for the report header.
5. **Quick ranges.** Last 7 / 30 / 90 nights / All, measured back from the file's last night. These write the same start/end dates the report and CSV use, so the charts can never show a different period than the PDF.
6. **Charts** for the selected range (§8).
7. **Parsed Nights** table — first 12 rows on screen; the report and CSV cover all of them.
8. **Generate Therapy Report** opens the printable document; **Download Daily CSV** exports every night in range.

Upload failure messages

Message	Trigger
File is *N* MB — the limit is 8 MB.	Oversized file
The file is empty.	No content
No usable session records found. *N* line(s) could not be read — check this is a REPORTFL.TXT session log.	Parsed, but zero valid records
More than one serial number in this file	Records reference different serials; a report covers one device

8. Charts

Four charts, matching the report's series and thresholds, rendered with recharts. Loaded lazily — the ~300 kB chart bundle only downloads once a file has been parsed.

Chart	Mark	Encoding
Usage (hours)	Bars	Green ≥ 4 h; red with a diagonal hatch below 4 h; dashed 4 h reference line
Set Pressure	**Step line**	A setting that holds for weeks is a level, not a nightly magnitude — bars merged into a solid block and hid when it changed
AHI	Bars	Single hue, dashed reference line at AHI 5
Mask Leak	Bars	Single hue, dashed reference line at 24 L/min

Every chart has a hover tooltip giving that night's exact value, date and session count. A metric with no data in range shows “No values recorded for this metric in the selected range” rather than an empty axis, which reads as a rendering fault.

> **Accessibility note that must not be regressed.** The clinical green/red usage convention fails colour-vision separation on its own (ΔE 4.1 under deuteranopia, measured with the palette validator). It is never the only cue: the 4 h threshold line gives every bar a readable position, short nights carry a diagonal hatch, the legend names both states, and the Parsed Nights table carries compliance in words. Keep at least one non-colour cue if you restyle these.

> **Global CSS trap.** `styles.css` sets `svg { fill:none; stroke:currentColor; stroke-width:1.8 }` for the icon set. Chart marks that do not declare their own stroke *inherit* it — on a 2 px bar that dark stroke swamps the fill and every series renders near-black. A reset inside `.recharts-wrapper` neutralises it. If you add charts elsewhere, keep them inside that wrapper or repeat the reset.

9. Report generation (PDF)

`therapyReport.js` builds a self-contained printable HTML document and opens it in a new window with a Print / Save-as-PDF button. Charts are hand-rolled inline SVG rather than a charting library: the window has no bundler, printing must not depend on a JS runtime having finished laying anything out, and it keeps the module free of the chart bundle.

It is shared by both data paths. The only difference is the `pressureSource` flag (§6.2) and a provenance line naming the uploaded file. If the browser blocks the pop-up, the app says so explicitly rather than failing silently.

10. Settings screen & the push pipeline

Route `/settings`. **One common screen serves both CPAP and BiPAP devices** — nothing is hidden or disabled based on device type.

Draft / committed editing

The user edits a *draft*. *Committed* values are what the device last confirmed. The floating apply bar appears only

believed to hold.

Flow states

```
Idle ──▶ Confirming ──▶ Sending ──▶ Idle          (device ACKed)
                                └──▶ Failed ──▶ Idle  (draft reverted)
```

The Confirming dialog lists exactly the changes that will be sent. The Sending dialog is non-dismissable but offers **Stop waiting**, which closes it and explains that the server has the change and will deliver it when the device next connects.

What gets sent

Only the groups that actually changed. Changing the humidifier must not rewrite the ramp block.

Group	Fields
`COMFORT`	rampTime, humidifier, tubeType, maskType
`OPTIONS`	iMode, leakAlert, sleepMode
`CPAP` / `AUTO`	Ramp and A-Flex, sent with the current pressures — the device rewrites the block whole

Server contract

```
PATCH /devices/{serial}/settings
{ "CPAP": null, "AUTO": null, "S": null, "ST": null, "T": null,
  "VAPS": null, "CONFIG": null,
  "COMFORT": { "rampTime": 20, "humidifier": 5, "tubeType": 1, "maskType": 1 },
  "OPTIONS": { "iMode": true, "leakAlert": true, "sleepMode": false } }
      ↓  response carries commandId
GET /devices/{serial}/commands/{commandId}/status   →  ACKED | NACKED | TIMEOUT
```

A `NO_CHANGE` response means the server saw nothing to write; the draft is treated as already matching the device. The client polls for **20 seconds** (reduced from the API default of 40, which held a blocking dialog too long).

Outcome	User sees
`ACKED`	“Settings applied” — the device confirmed *N* changes
`NACKED`	“The device rejected the change” — values reported invalid, rolled back
`TIMEOUT`	“The device didn't confirm” — server accepted it, device never acknowledged; may be offline or asleep
HTTP error	Classified by status (§11)
Network failure	“Couldn't reach the server”

11. Messaging: the notification frame

All user-facing messaging goes through one surface. `NotifyProvider` is mounted **above the router**, so a message raised anywhere — app shell, HCP portal, login, onboarding — lands in the same frame in the same corner. Screens no longer each invent their own `alert()` / toast / inline banner; there are now **zero** `alert()` calls in application screens.

Tone	Auto-dismiss	Use
success	4 s	An action completed
info	5 s	Something is under way
warning	8 s	Partial success, or a caveat
error	**Never** — stays until dismissed	A message about changes that did not save must not vanish before it is read

A maximum of four are visible; an identical repeat refreshes the existing card rather than stacking noise. Retryable errors carry a **Try again** button; non-retryable ones deliberately do not, so the user is never invited into a retry that is guaranteed to fail.

Error classification

`apiError.js` converts any thrown failure into title / message / technical detail, so the same underlying problem reads identically on every screen. The API layer attaches `status` and the response body to its errors specifically to make this possible.

Condition	Title	Message	Retry
Offline / fetch failed	Couldn't reach the server	You appear to be offline. Check your connection and try again.	Yes
400 / 422	The server rejected those values	The server's own text — it normally names the offending field	No
401 / 403	Your session has expired	Sign in again to continue. Your unsaved changes were not sent.	No
404	Not found on the server	The server has no record of *subject*. It may have been removed, or the serial may be wrong.	No
409	Someone else changed this first	Refresh to load the current values, then apply your change again.	Yes
429	Too many requests	The server is rate-limiting this device. Wait a moment and try again.	Yes
5xx	The server had a problem	It could not *action*. This is on our side — try again shortly.	Yes
Anything else	Couldn't *action*	The raw error text	Yes

12. Complete message catalogue

Every message the app raises through the frame, by screen.

Screen	Tone	Message
Any (device load)	error	Classified failure loading a device, with retry. Previously a silent `console.warn` that left every screen on “Loading therapy data…” forever.
Settings	success	Settings applied — the device confirmed *N* change(s)
Settings	error	Couldn't apply settings / device rejected / didn't confirm — with retry where sensible
Settings	info	Still sending in the background — the server has the change for *serial*
Device dashboard	error	Classified push failure; device did not apply the settings
Admin settings editor	error	Failed to load / save these settings, with retry
Admin patients	warning	Patient created, but the welcome email failed — copy the login link and send it manually
Admin patients	error / success	Onboarding failure (classified) · Login link copied
Reports	error	The report window was blocked — allow pop-ups, then generate again
Reports	success	Share link sent
Devices	success / warning	Sync complete · Device disconnected
Trends	info	Preparing export
Help & Support	info / success	Running *diagnostic* · Support ticket raised
Preferences	success	*Action* triggered — the device will apply it on next sync
Create organization	error	Classified registration failure
HCP portal	error	Sign-in failed — those credentials were not recognised

13. BLE protocol reference module

`protocol/bleProtocol.js` is a complete, tested implementation of the CPAP/BiPAP BLE hex protocol Rev 3.1 — FULL_SYNC, UPDATE and NACK frames, XOR checksums, range validation and engineering-unit conversion. **It is not used by the web app** (§1) and lives outside `src/` for that reason. It is there for whoever owns the server or firmware side.

Run `node protocol/bleProtocol.test.mjs` — 47 assertions, no dependencies.

Errors found in the Rev 3.1 specification

The byte map is internally consistent and the codec matches it exactly. Three of the document's hand-written worked examples do not:

Example	Document states	Correct

CPAP FULL_SYNC	72 bytes, `CK 0x60`	70 bytes, `CK 0x6B` (two extra `00` padding bytes)
BiPAP FULL_SYNC	`CK 0x24`	`CK 0x2F`
BiPAP `#4_ST` UPDATE	`CK 0x56` (ACK `0x57`)	`CK 0x5A` (ACK `0x5B`) — the XOR stops before SENS and RISE_TIME

The CPAP `#1` UPDATE, its ACK and the NACK example all validate, which confirms the checksum rule itself is right.

> **Packet discrimination.** FULL_SYNC has no DIR byte, so byte 1 is its packet type `0x01` — identical to an UPDATE ACK whose DIR is `0x01`. Byte 2 does not separate them either (device type BIPAP and packet type UPDATE are both `0x02`). **Length is the only reliable discriminator.** If firmware ever changes a packet length, this is what breaks.

14. File map

Path	Responsibility
`src/services/reportFileParser.js`	REPORTFL.TXT parsing, validation, nightly aggregation, CSV export
`src/pages/AdminDataUpload.jsx`	Upload screen: drop zone, summary, warnings, ranges, actions
`src/components/ReportCharts.jsx`	The four on-screen charts
`src/services/therapyReport.js`	Printable therapy report (HTML + inline SVG charts)
`src/pages/Settings.jsx`	Device comfort & configuration screen
`src/pages/AppPreferences.jsx`	App-level preferences (units, notifications, account, appearance), route `/preferences`
`src/context/NotifyContext.jsx`	Notification frame — provider, queue, card rendering
`src/services/apiError.js`	Error classification into user-facing language
`src/services/respireeApi.js`	Server API: devices, telemetry, settings groups, command polling
`src/context/TherapyContext.jsx`	Active device state and the shared fetch
`protocol/`	BLE protocol reference implementation + tests + README (not used by the app)

15. Running & testing

```
npm install
npm run dev      # Vite dev server, proxies the API
npm run build    # production build
node protocol/bleProtocol.test.mjs   # 47 protocol assertions
```

There is no test runner wired into the project; the protocol suite is a plain Node script with no dependencies and exits non-zero on failure, so it drops straight into CI.

16. Open items requiring a decision

Item	Current behaviour	Needed
Leak codes ≥ 120	Treated as status codes, excluded from leak statistics, surfaced to the operator	Firmware to confirm the encoding. One constant changes if they are real readings.
COMFORT / OPTIONS on CPAP devices	The settings screen offers them on every device; the server currently answers *"Mode COMFORT is not applicable to device type CPAP"* and the frame reports it	Backend to accept those groups for CPAP-registered devices, or a product decision to the contrary.
Mask vs tube counts	UI offers 3 masks and 2 tubes; codec accepts mask 1–3	The protocol says mask 1–2 and tube 1–3 — the two rows look transposed in one document. Confirm which is authoritative.

RISE_TIME valid set	Any multiple of 50 ms in 100–650 accepted	The spec's "valid" list omits 300 ms, yet its own worked example uses it. Confirm the real constraint.
Therapy-running lock	Settings reads `deviceData.therapy_running`; nothing sets it, so the locked state never triggers	A therapy-running signal from the server.